`

Assignment

Aim: -

Simulation of Interfacing of ultrasonic sensor and buzzer

Theory:-

The HC-SR04 ultrasonic sensor measures distance by emitting 40 kHz sound pulses and timing their echo to calculate range. **Piezo buzzers** use the reverse piezoelectric effect to convert an applied alternating voltage into mechanical vibrations that produce sound. In Tinkercad Circuits, you simulate this by wiring the sensor's Trig/Echo pins to Arduino digital I/O, connecting the buzzer to an output pin, and using the built-in distance slider or virtual object tool to emulate proximity—all within a browser-based virtual Arduino environment.

Theory of Ultrasonic Sensor

Working Principle

The ultrasonic sensor operates on the **time-of-flight principle**: it emits a high-frequency acoustic pulse and measures the time interval until the echo returns. Distance is then computed as:

D=t*v(sound)/2

where ttt is the round-trip time of the pulse and vsound ,vsound is the speed of sound (≈ 343 m/s in dry air at 20 °C). The HC-SR04 module emits several 40 kHz bursts when it receives a 10 µs trigger pulse, then raises its Echo output pin high for the duration until the reflected waves are detected. An Arduino (or other microcontroller) reads this Echo pulse width—typically via a function like `pulseIn()`—to get the time in microseconds.

Key Components

- **Transducers:** Both the transmitter and receiver in the HC-SR04 are piezoelectric ceramic elements that convert electrical signals into sound waves and vice versa.
- **Pins:** Four pins (VCC, Trig, Echo, and GND) allow easy connection to development boards.

Theory of Piezoelectric Buzzer

Piezoelectric Effect

Inside a piezo buzzer is a ceramic disk that bends when an alternating voltage is applied. This mechanical deformation oscillates at audio frequencies, producing sound waves. Compared to magnetic buzzers, piezo versions draw less current and have simpler drive requirements, making them ideal for low-power or battery-powered projects.

Driving the Buzzer

A microcontroller drives a piezo buzzer by toggling a digital output pin on and off. In a distance-alert application, the code switches the buzzer pin high and low in a loop based on the measured distance, creating beeps whose repetition rate increases as an object gets closer.

Tinkercad Simulation Environment

Virtual Wiring

Tinkercad Circuits provides virtual Arduino and sensor parts. You place an HC-SR04 and a piezo buzzer in the workspace and click each pin to drag wires directly to the Arduino headers (VCC → 5 V, GND → GND, Trig → D9, Echo → D10, buzzer "+" → D8, buzzer "–" → GND).

Distance Emulation Tools

When you start the simulation and click on the HC-SR04, a distance slider appears. Dragging this slider changes the reported distance value so you can observe how your sketch reacts. In some example circuits, you can also click and drag a virtual "object" icon toward the sensor to mimic a moving obstacle, watching the Serial Monitor update and the buzzer beep in real time.

Connection:-

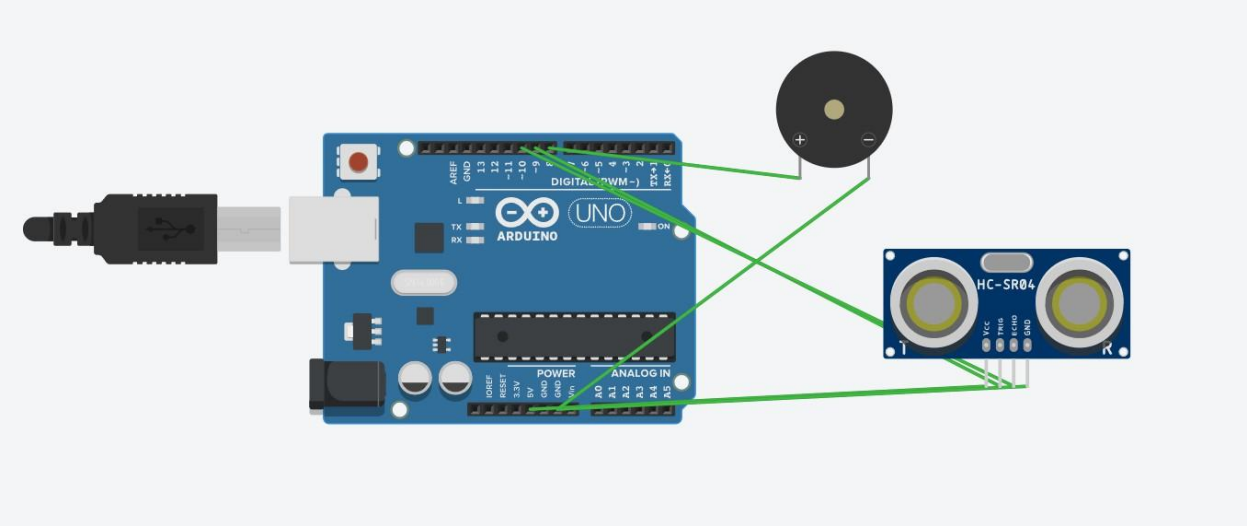
Component	Module Pin	Arduino Pin
HC-SR04 Sensor	VCC	5 V
HC-SR04 Sensor	GND	GND
HC-SR04 Sensor	Trig	D9
HC-SR04 Sensor	Echo	D10
Piezo Buzzer	"+"	D8
Piezo Buzzer	"–"	GND

Programs

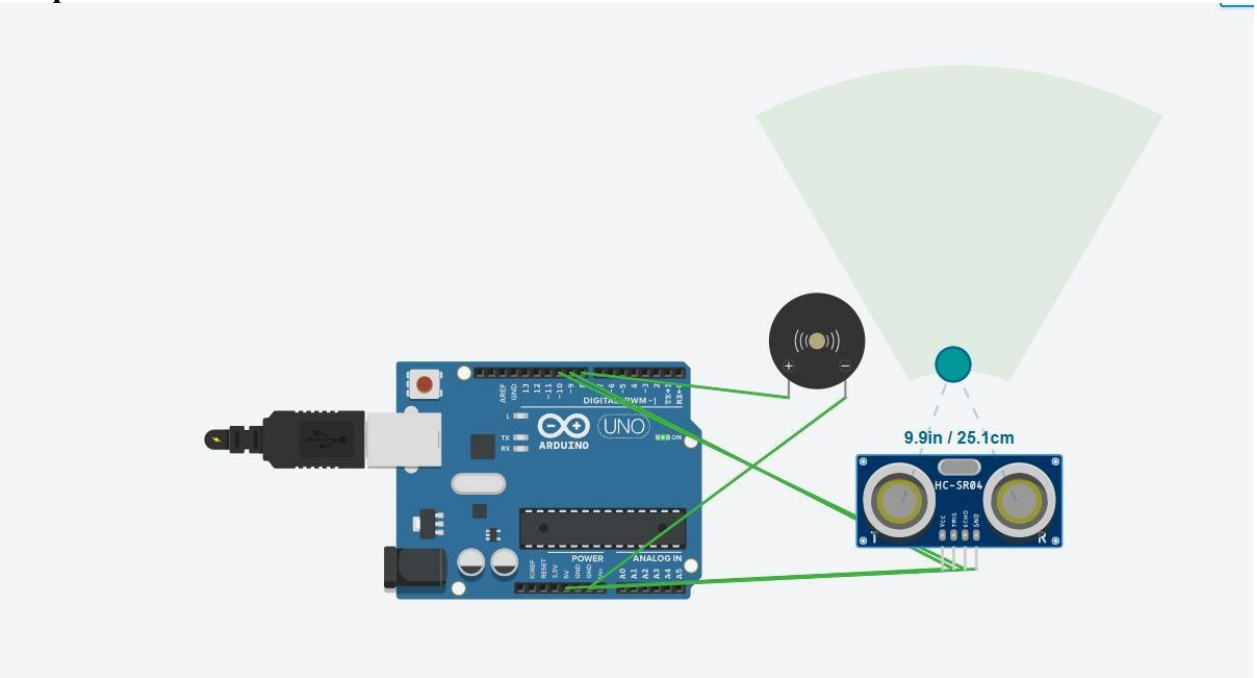
```
const int TRIG_PIN  = 9;
const int ECHO_PIN  = 10;
const int BUZZER_PIN = 8;

long duration;
int distanceCm;

void setup() {
  pinMode(TRIG_PIN, OUTPUT);    // HC-SR04 trigger
  pinMode(ECHO_PIN, INPUT);     // HC-SR04 echo
  pinMode(BUZZER_PIN, OUTPUT);  // Piezo buzzer
  Serial.begin(9600);
}

void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH);
  distanceCm = duration * 0.034 / 2;
  Serial.print("Distance: ");
```

```
  Serial.print(distanceCm);
  Serial.println(" cm");


  if (distanceCm < 30) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(100);
    digitalWrite(BUZZER_PIN, LOW);
    // Ensure non-negative delay
    delay(max(50, 150 - distanceCm * 5));
  } else {
    delay(200);
  }
}
```

Circuit:-



Output:



Conclusion:

Simulating the HC-SR04 and piezo buzzer in Tinkercad lets you prototype and test a proximity-alert system quickly and safely, using the same wiring and code you'll deploy on real hardware. The built-in distance slider and draggable object tools make it easy to tune thresholds and observe behavior instantly. This hands-on virtual approach deepens your understanding of time-of-flight sensing and actuator control, so you can confidently move from simulation to a physical build—and then expand your project with displays, multiple sensors, or wireless alerts.